

La Couche de Transport dans le modèle de l'IETF (partie 1 : les bases)

Thierry DESPRATS

Université de Toulouse

FSI – Dpt Info // IRIT- SIERA

Thierry.Desprats@irit.fr

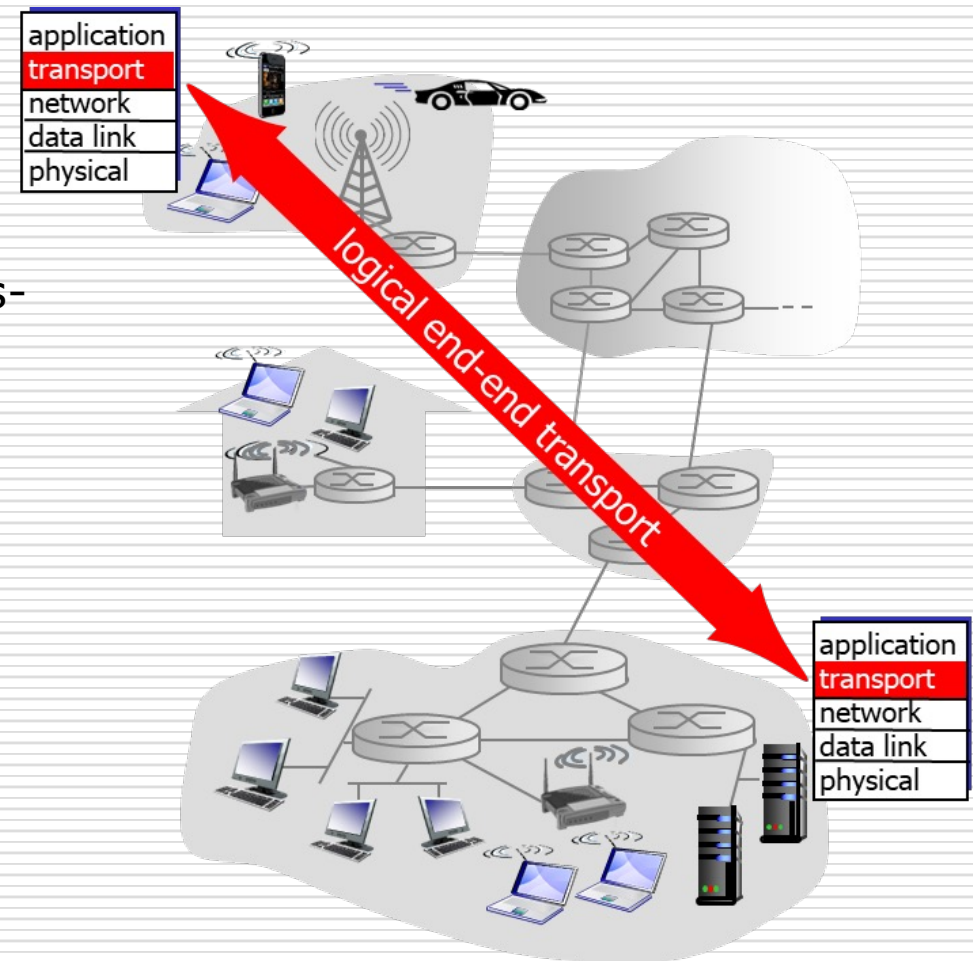
Agenda

- ☐ La couche de Transport
 - Objectifs
 - Architecture IETF
- ☐ Notion de port
- ☐ UDP
 - Caractéristiques
 - Format des datagrammes
- ☐ Fiabilisation des transferts
- ☐ TCP
 - Caractéristiques
 - Gestion des connexions
 - Transfert des données
 - Format des segments

Couche de Transport (1/3)

□ Objectifs

- Offrir un service de transfert de données de bout en bout en faisant abstraction à la fois des problèmes d'acheminement et de l'interconnexion des réseaux sous-jacents.
- Communication **logique entre processus applicatifs** s'exécutant sur des hôtes différents (contrairement à la couche réseau qui offre une communication logique entre hôtes !)
- Transport sur les équipements d'interconnexion ?



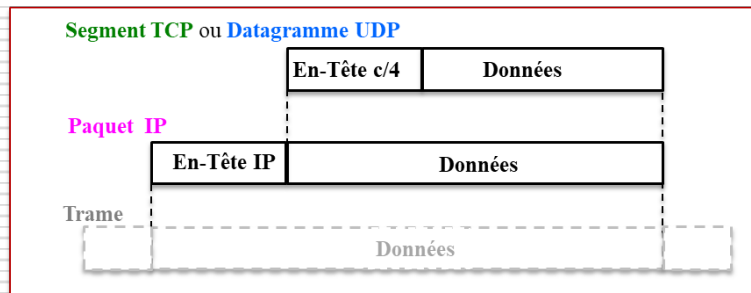
Couche de Transport (2/3)

□ Architecture IETF :

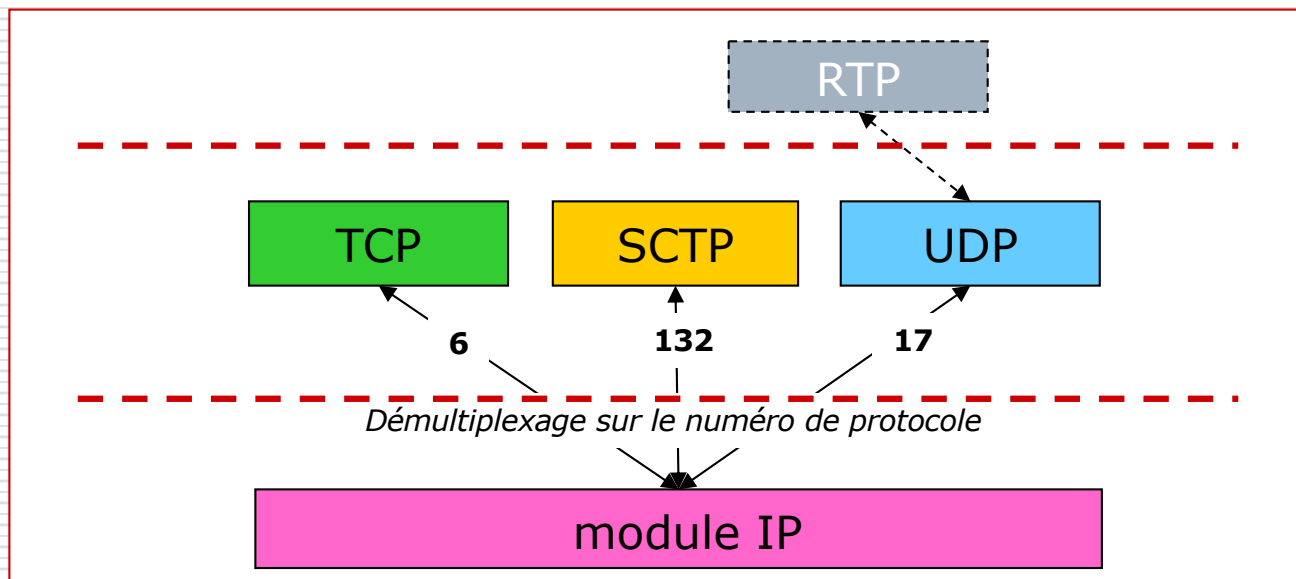
- Services supportés par les 2 protocoles historiques standardisés :
 - service en mode non connecté, remise non fiable
 - *UDP (User Datagram Protocol)* [RFC 768/STD 6]
 - service en mode connecté, remise fiable
 - *TCP (Transmission Control Protocol)* [RFC 793/STD 7]
- Autres services :
 - protocole directement situé en couche de Transport :
 - *SCTP (Stream Control Transmission Protocol)* [RFC 4960/Proposed Std]
 - WG « Signaling Transport » de l'IETF (SIGTRAN) à partir de 99 : initialement, dédié au transport de messages de signalisation sur une infrastructure IP pour les réseaux téléphoniques commutés publics (PSTN – Public Switched Telephone Network)
 - D'autres applications possibles car gestion de la multi-domiciliation (échange initial des adresses IP)
 - s'appuyant sur les protocoles standards de la couche de Transport :
 - *R(C)TP (Real-Time (Control) Transport Protocol)* [RFC 3550/STD 64]
 - WG « Audio Video Transport » de l'IETF (AVT) à partir de 95/96 : flux à contrainte temps réel sur IP
 - *QUIC (A UDP-Based Multiplexed and Secure Transport)* [RFC 9000/Proposed Std mai 2021]
 - UDP + TLS1.3 = = > HTTP/3 nouveau WG de l'IETF (2016)

Couche de Transport (3/3)

□ Encapsulation dans IP :



□ Démultiplexage/remise par IP :



Notion de Port (1/4)

□ Identification des Processus d'application :

- L'adresse IP permet de désigner de façon unique un équipement sur un réseau internet.
- Généralement plusieurs processus d'application s'exécutent sur un host et peuvent émettre ou recevoir des données à partir des services de Transport :
= = > NÉCESSITÉ pour les services de Transport d'identifier le processus à qui sont destinées les données transmises.

□ Numéro de port :

- Nombre entier associé à un processus d'application : il permet la désignation unique de celui-ci parmi un ensemble de processus s'exécutant sur un même équipement.
- Plus précisément, identifie un processus d'application utilisateur d'un protocole de Transport précis : « port TCP », « port UDP » et « port SCTP »
- Codage sur **16 bits**

Notion de Port (2/4)

□ Numérotation des ports :

■ Gestion par l'**IANA** :

- Historiquement RFC 1700
- Base de données en ligne [RFC 3232] début 2002 :
<http://www.iana.org/assignments/port-numbers>

■ Trois catégories :

□ Ports « Well-known »

- Définissent des points de contacts reconnus, réservés à des services standardisés
- Réservés aux processus système (démons) ou aux programmes exécutés par des utilisateurs privilégiés. Un administrateur réseau peut néanmoins lier des services aux ports de son choix.
- Intervalle de valeurs : [0..1023]
- Exemples : 20-21(ftp) 23(telnet) 25(smtp) 53(DNS) 80(http) 101(pop3)

□ Ports « Registered »

- Définissent des points de contacts de services enregistrés auprès de l'IANA
- Manipulés par des processus aux droits ordinaires
- Intervalle de valeurs : [1024..49151]
- Exemples : 1986(Cisco license management) 13782(Veritas NetBackup)

□ Ports « Private and/or Dynamic »

- Utilisables sans contrainte (Serveur, Client, P2P, streaming, etc...)
- Intervalle de valeurs : [49152..65535]

Notion de Port (3/4)

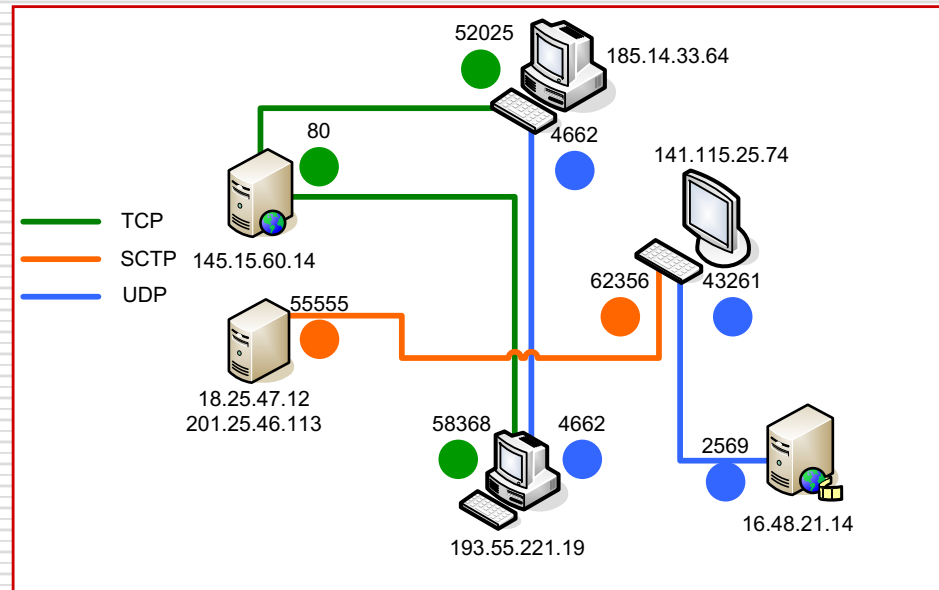
□ Identification d'associations d'application :

- Une association entre deux processus d'application peut être décrite à chaque extrémité de façon unique par un quintuplet :

- Adresse(s) IP locale(s),
- numéro de port local,
- Adresse(s) IP distante(s),
- numéro de port distant,
- type du protocole de transport (TCP, UDP, SCTP)

= = > **concept de sockets (locale et distante)**

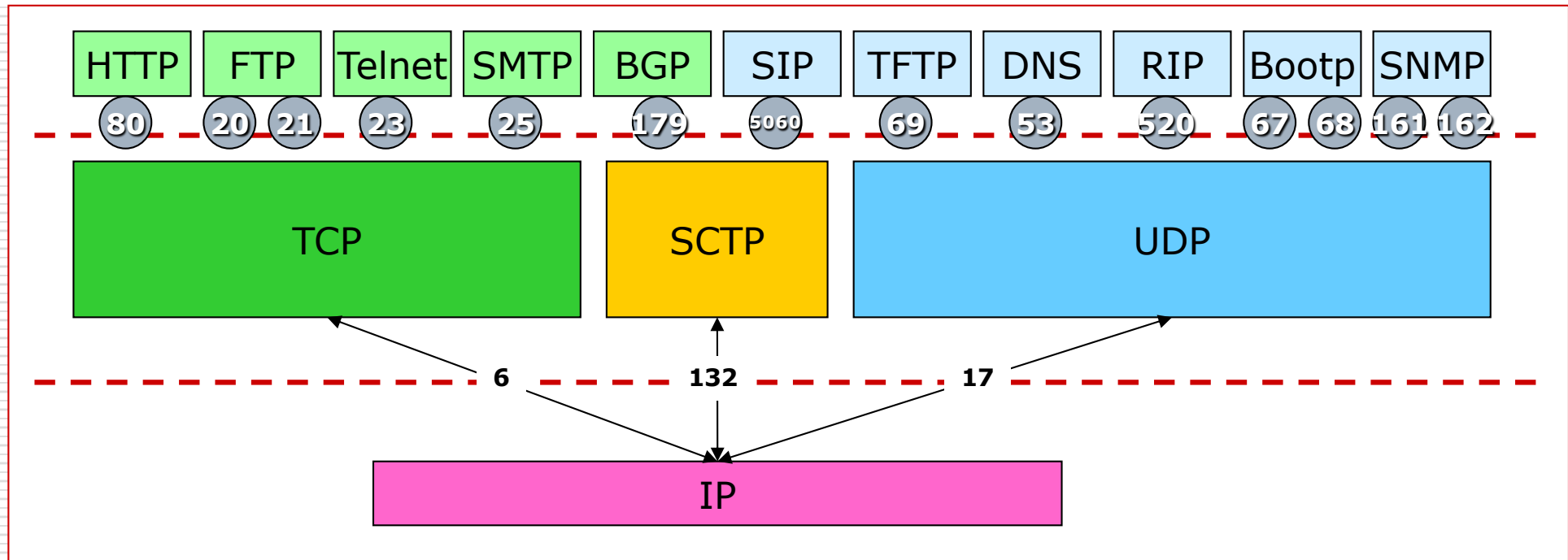
- Illustration :



<u>TCP</u>	145.15.60.14	80	185.14.33.64	52025
<u>TCP</u>	145.15.60.14	80	193.55.221.19	58368
<u>UDP</u>	185.14.33.64	4662	193.55.221.19	4662
<u>UDP</u>	141.115.25.74	43261	16.48.21.14	2569
<u>SCTP</u>	(18.25.47.12, 201.25.46.113)	55555	141.115.25.74	62356

Notion de Port (4/4)

□ Démultiplexage Transport :

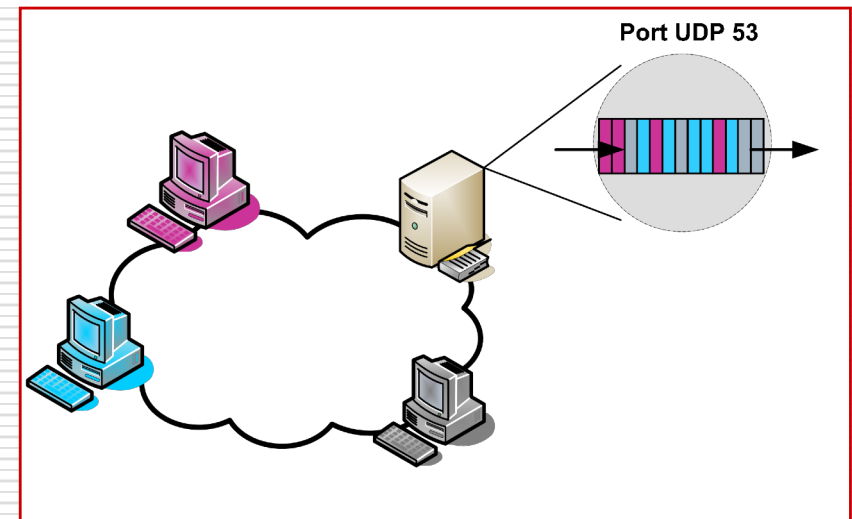


□ Différents modes opératoires possibles :

- Protocoles : mono vs multi (ex TCP et UDP)
- Services : mono vs multi
- Processus et thread : mono vs multi

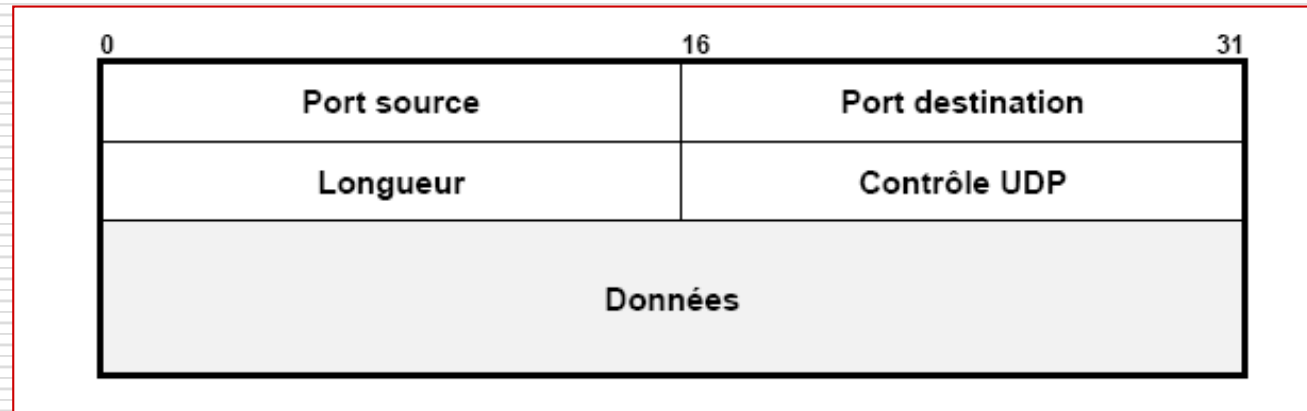
UDP (1/2) Généralités (1/1)

- ❑ Caractéristiques générales de **UDP** :
 - ❑ N'apporte que peu de fonctionnalités supplémentaires à IP :
 - Identification des processus d'applications par ports
 - Contrôle éventuel de l'intégrité des données par *checksum*.
 - ❑ Service de transfert de données non fiable :
 - Pertes, duplications, retards, déséquences possibles
 - Ni contrôle de flux, ni contrôle d'erreurs, ni retransmission**=> service *best effort***
- ❑ Port UDP :
 - ❑ *Bufferisation* FIFO des messages : chaque paquet est traité indépendamment des autres (sans connexion)
- ❑ Exemples d'applications :
 - ❑ Résolution de noms (DNS), synchronisation d'horloge (NTP), supervision de réseau (SNMP), signalisation voix sur IP (SIP)...



UDP (2/2) Protocole (1/1)

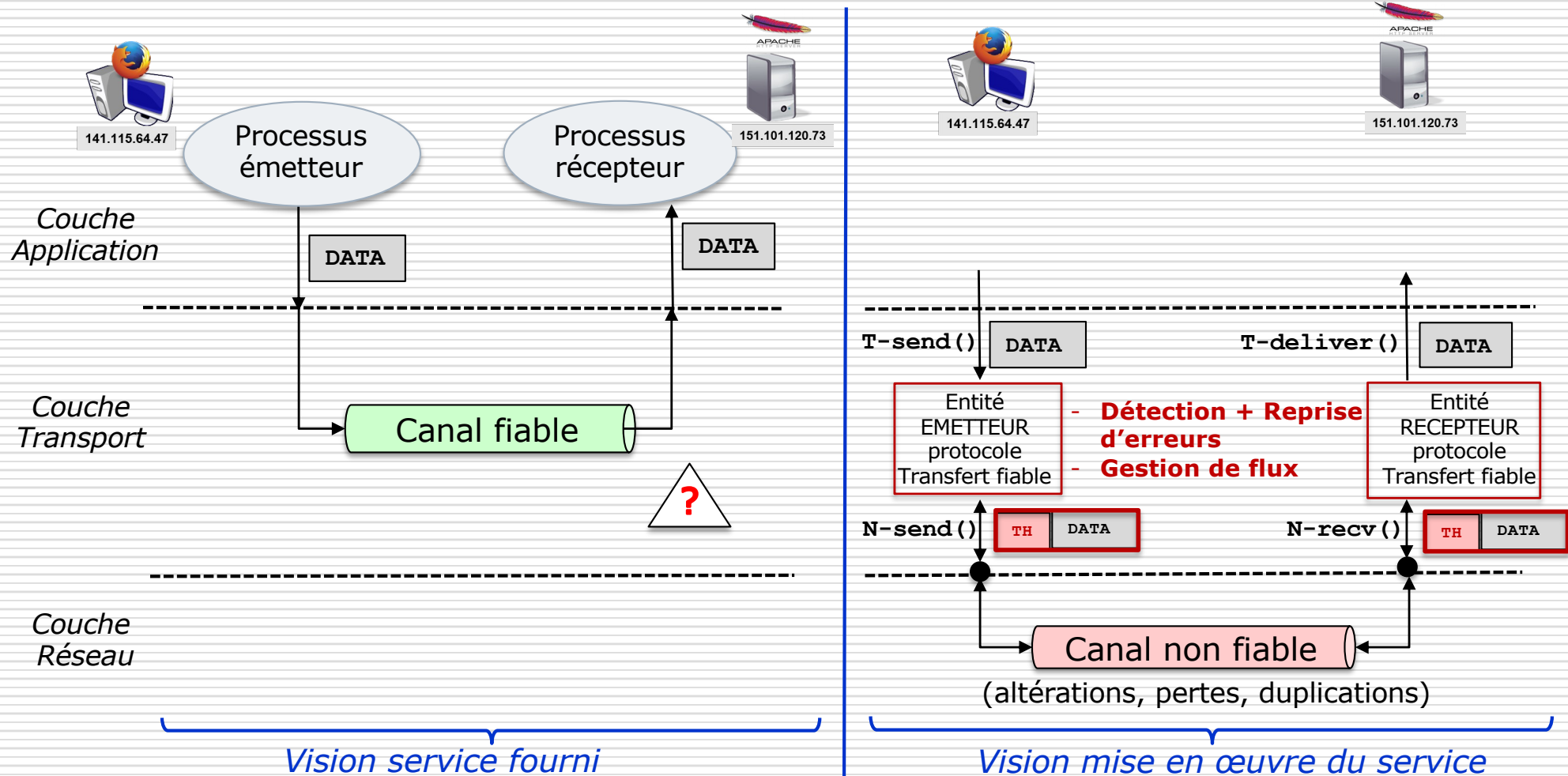
□ Format d'un datagramme UDP:



- Champ Port Source (16 bits) :
 - Référence facultative (à zéro sinon) au processus émetteur.
- Champ Port Destination (16 bits) :
 - Référence au processus destinataire distant (utile au démultiplexage chez le récepteur).
- Champ Longueur (16 bits) :
 - Longueur totale en octets du message UDP (min = 8).
- Champ Checksum (16 bits) :
 - Somme de contrôle d'intégrité du paquet (calcul idem TCP)
 - Facultatif pour IPv4 (à zéro sinon), obligatoire pour IPv6.

Fiabilité des transferts (1/12)

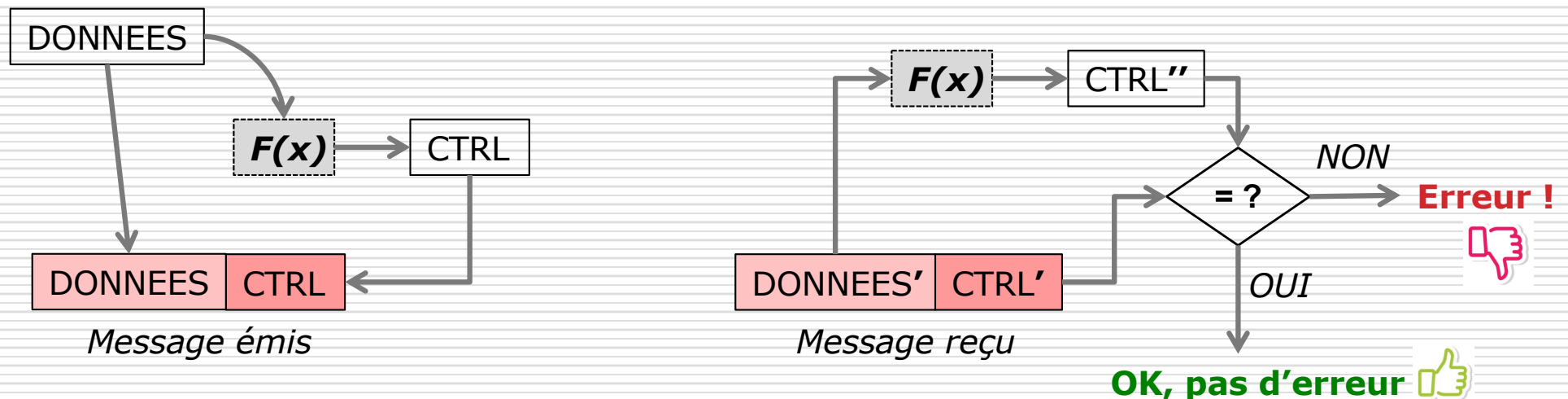
□ Principes de la fiabilisation



[Inspirée de Kurose and Ross]

Fiabilité des transferts (2/12)

- ❑ Bases de la détection d'erreurs (altérations)
 - Principe de base : ajout de bits de contrôle aux bits de données qui munit l'unité de données à envoyer d'une caractéristique qui sera vérifiée à la réception.

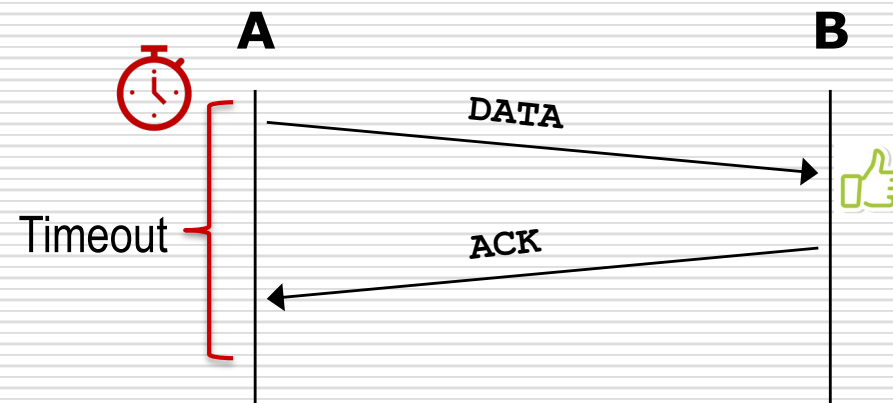


EMETTEUR ----- ➔ **RECEPTEUR**

- $F(x)$ est appelé code détecteur d'erreurs. Elle peut être implémentée par un CRC ou une somme de contrôle (*checksum*).

Fiabilité des transferts (3/12)

- ❑ Bases de la reprise sur pertes et erreurs
 - Méthode [ARQ](#) (*Automatic Repeat reQuest*) basée sur l'émission d'acquittements et la gestion de temporisateurs
 - ❑ Le récepteur envoie un acquittement lorsqu'un paquet est reçu sans erreur.
 - ❑ L'émetteur retransmet à l'expiration d'une échéance (timeout) tant qu'un acquittement n'est pas reçu
 - Illustrations :
 - ❑ Cas d'une opération normale

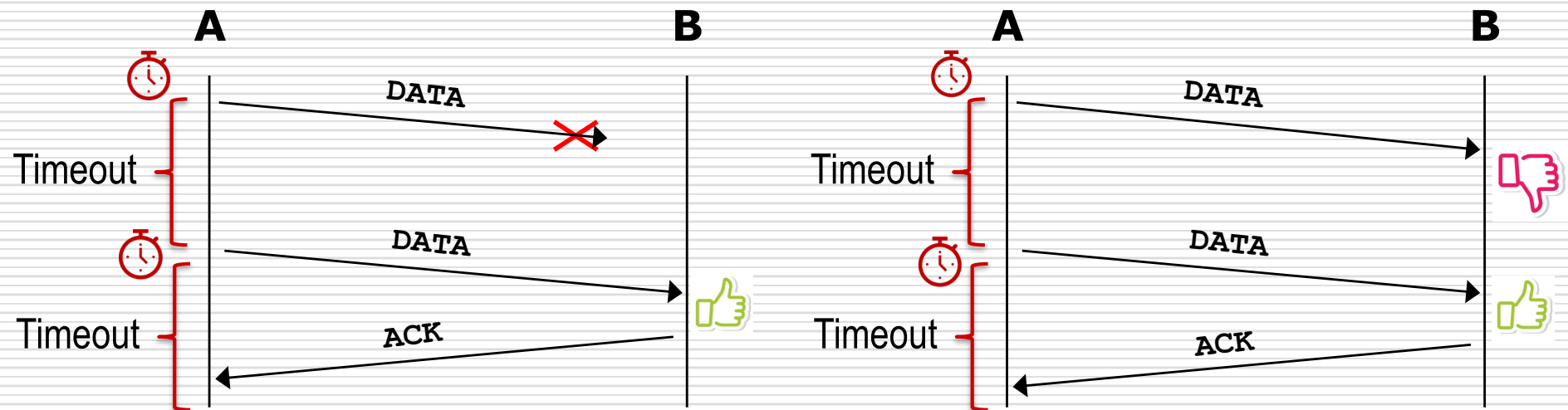


Fiabilité des transferts (4/12)

□ Bases de la reprise sur pertes et erreurs

■ Illustrations ARQ :

□ Retransmission sur perte ou erreur des données

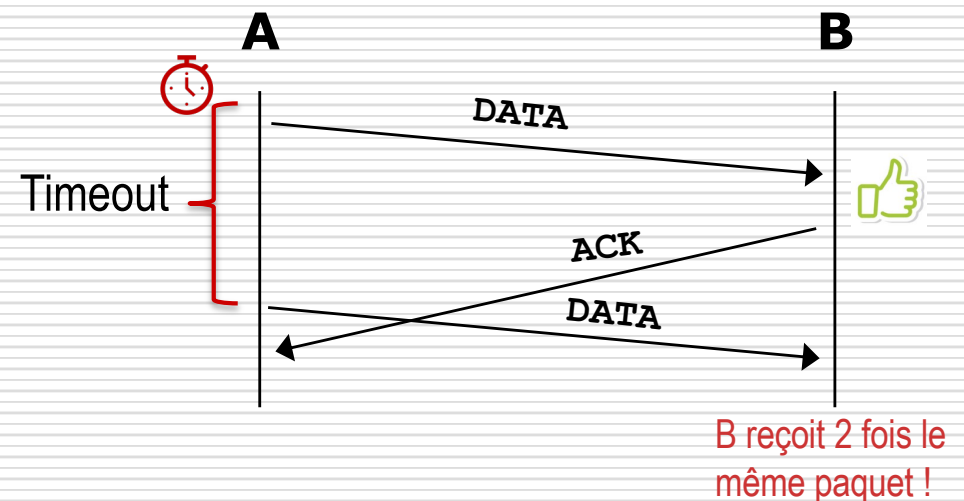
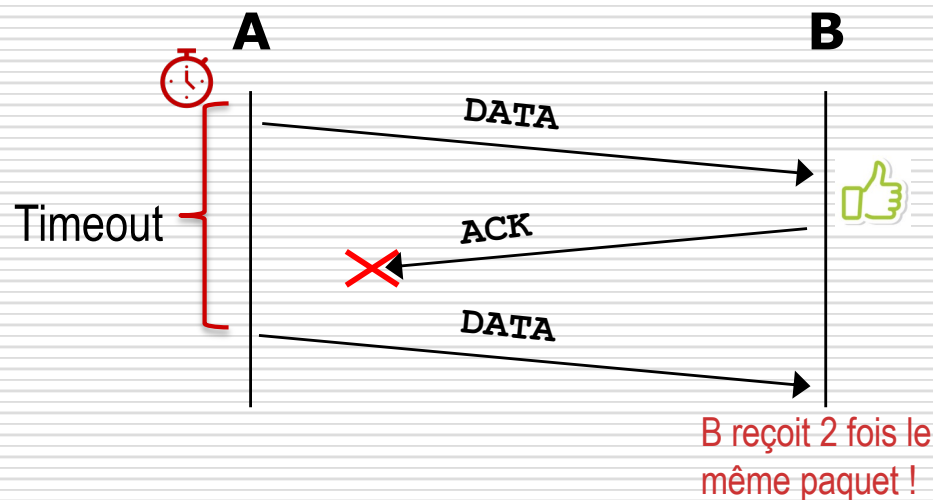


Fiabilité des transferts (5/12)

□ Bases de la reprise sur pertes et erreurs

■ Illustration du problème de gestion des doublons :

□ Perte ou retard d'un acquittement



■ Solution : Utiliser les numéros de séquence pour filtrer les doublons (données et acquittements)

==> En tête segment TCP : numéro de séquence, numéro d'ACK

Fiabilité des transferts (6/12)

□ Bases de la gestion d'un flux

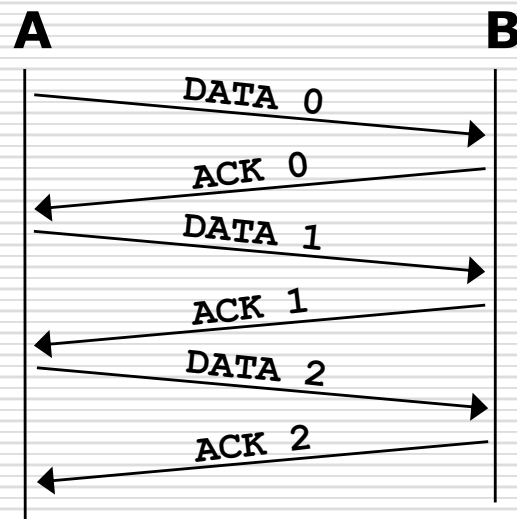
■ Politiques de contrôle du débit d'émission :

□ « Stop & Wait »

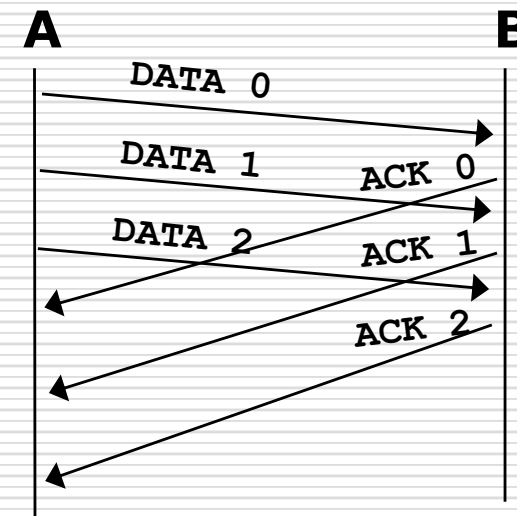
vs

« Fenêtre d'anticipation »

Sliding window (fenêtre glissante)



- Un paquet après l'autre
- Usage faible du canal
- Mémoires tampons de 1 paquet



- Pipeline de transmission
- Usage réseau amélioré
- Tailles buffers plus importantes

Fiabilité des transferts (7/12)

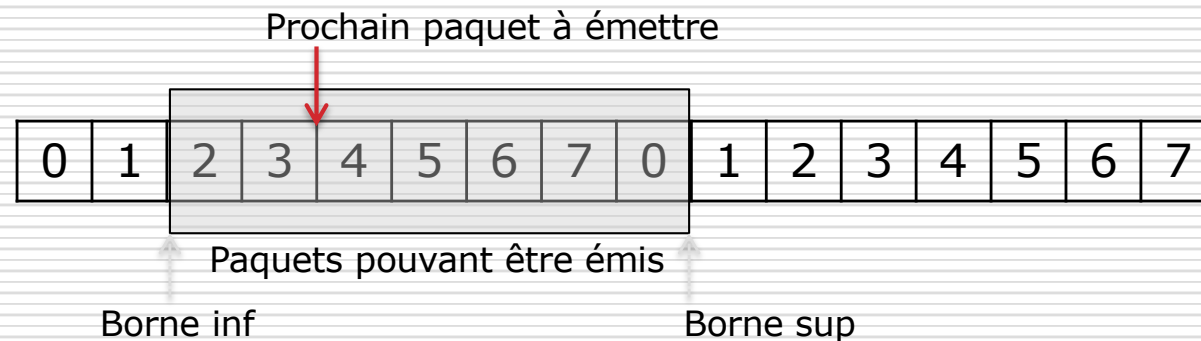
- Bases de la gestion d'un flux
 - Fenêtre Côté Emetteur (Ws) :
 - Contrôler le débit d'émission du flux en considérant 2 contraintes :
 - Exploiter au mieux le débit du réseau (*cf* contrôle de congestion)
 - Éviter les pertes chez le récepteur (*cf* contrôle de flux)
 - Solution : permettre l'anticipation :
 - L'émetteur peut émettre plusieurs paquets sans devoir attendre les acquittements de ceux précédemment émis
 - Le nombre total de paquets pouvant être émis dépend du crédit ou « taille de la fenêtre d'anticipation »
 - Bufferisation (en vue de retransmissions) et numérotation des paquets nécessaires
 - Les réceptions d'acquittements vont provoquer le décalage de la fenêtre

Fiabilité des transferts (8/12)

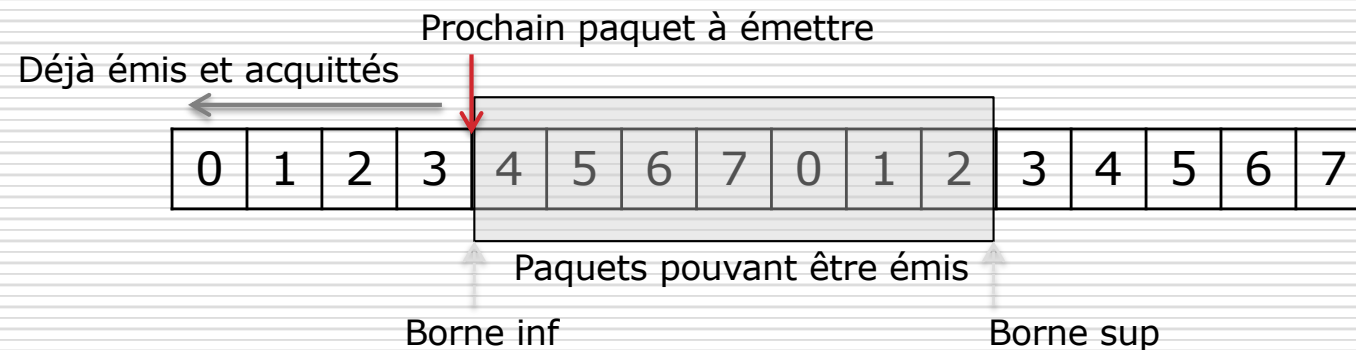
□ Bases de la gestion d'un flux

■ Fenêtre Côté Emetteur : illustration (crédit = 7, numérotation modulo 8)

Ws :



Ws après réception ack 2 et 3 :



Fiabilité des transferts (9/12)

□ Bases de la gestion d'un flux

■ Fenêtre côté Récepteur (W_r) :

- Contrôler le séquençement, possiblement minimiser les retransmissions

□ Principe :

- Définit les paquets pouvant être reçus, dans l'ordre ou non, selon le protocole de retransmission retenu
- Lors de la réception de paquets corrects, émission d'acquittement et possible décalage de la fenêtre

□ Alternatives protocolaires :

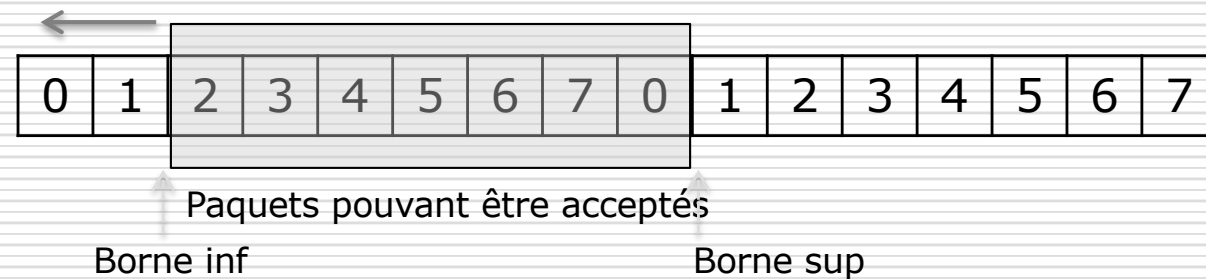
- Les acquittements peuvent être soit individuels, soit cumulatifs
- Les données hors séquence reçues correctement peuvent être :
 - globalement rejetées (taille fenêtre = 1) → protocole **Go Back N** qui implique une retransmission de TOUTES les données hors séquence déjà émises
 - bufferisées → protocole **Selective Repeat** qui permet de ne retransmettre individuellement que les données erronées ou perdues.
Donc les données hors séquence correctement reçues sont bufferisées par le récepteur dans l'attente des données manquantes

Fiabilité des transferts (10/12)

□ Bases de la gestion d'un flux

■ Fenêtre Côté Récepteur : illustration (crédit = 7, numérotation modulo 8)

Wr : Déjà reçus et acquittés



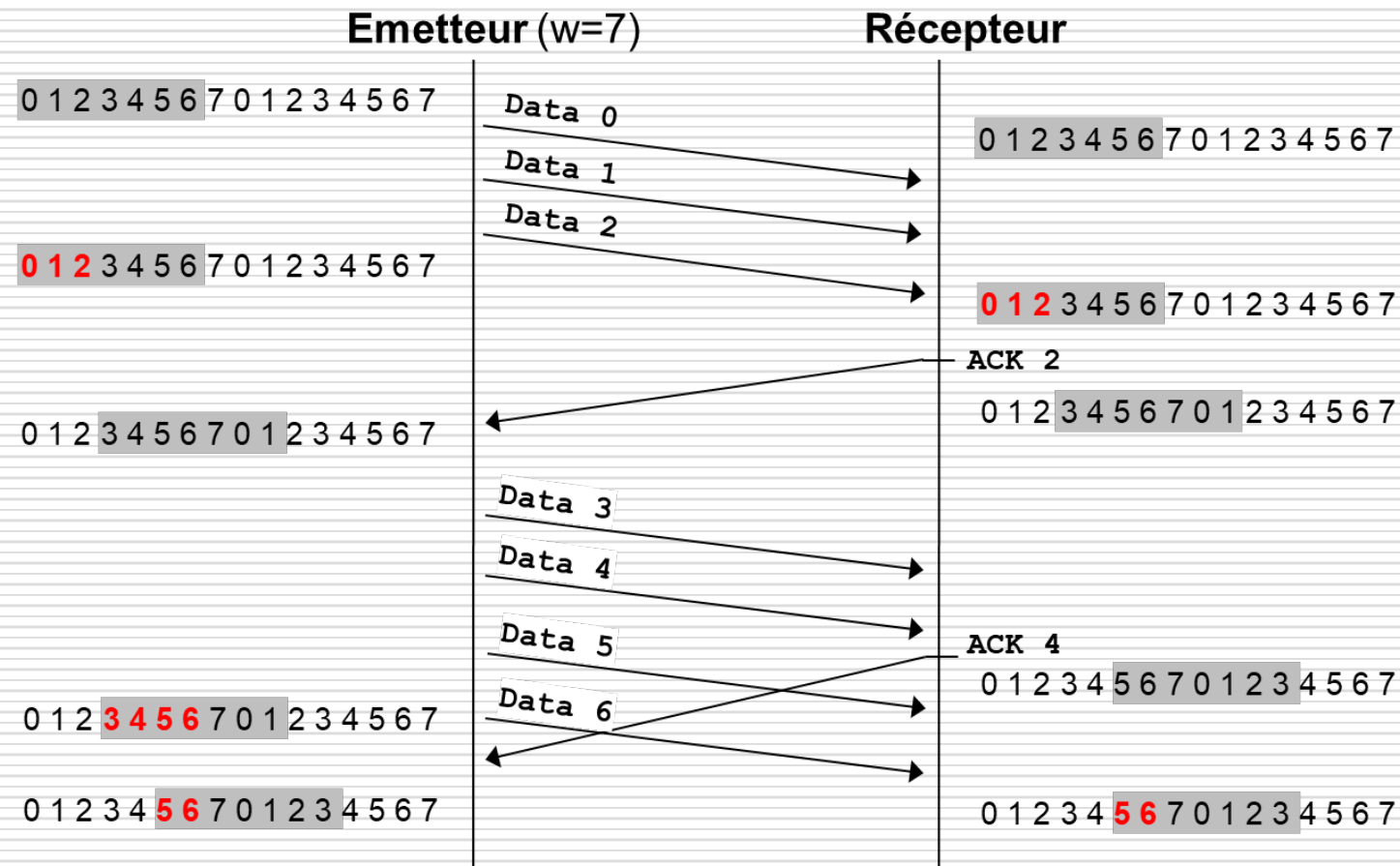
Wr après réception 2 et 3 et émission acks :



Fiabilité des transferts (11/12)

□ Bases de la gestion d'un flux

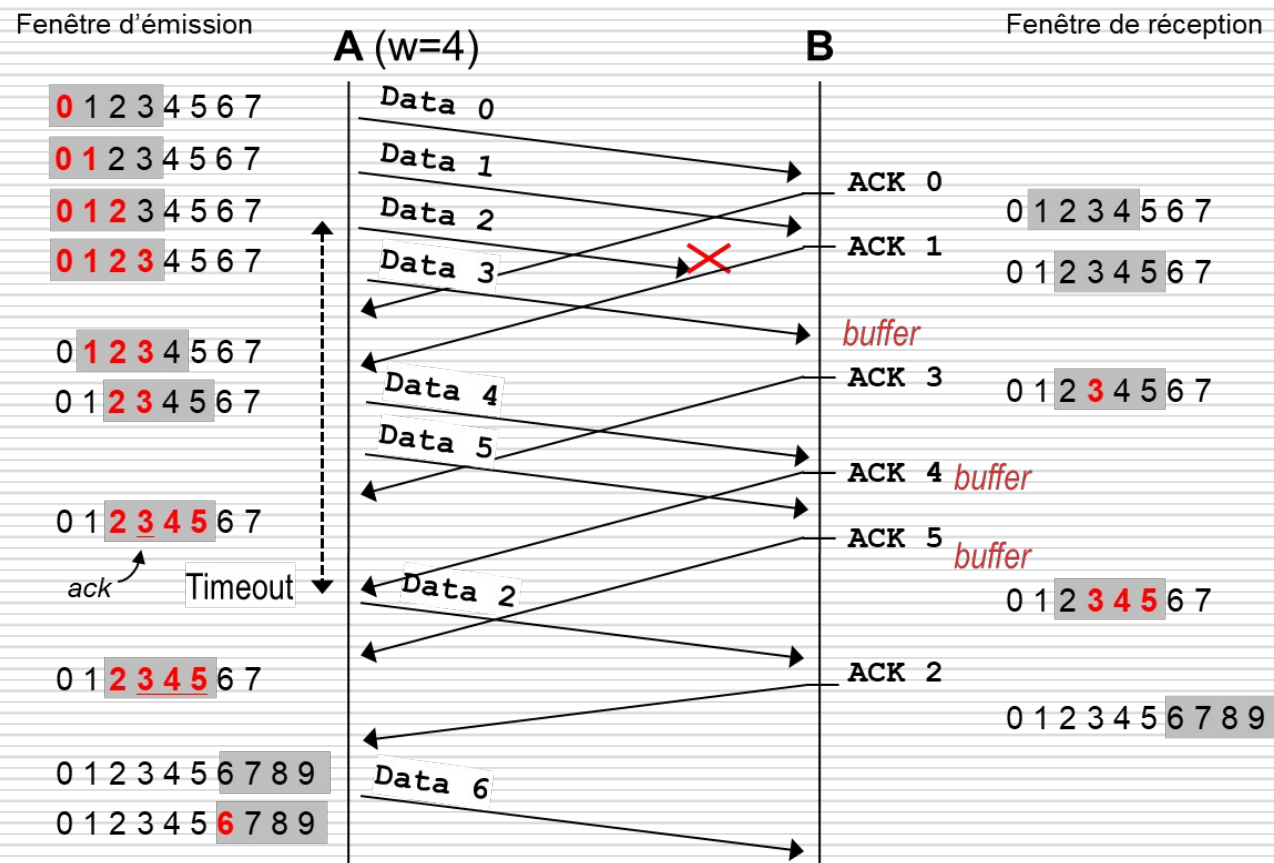
■ Scénario (crédit = 7, numérotation modulo 8, ACK cumulatifs)



Fiabilité des transferts (12/12)

□ Bases de la gestion d'un flux

■ Scénario anticipation et retransmission [Selective Repeat](#) (crédit = 4, numérotation modulo 8, ACK individuels)



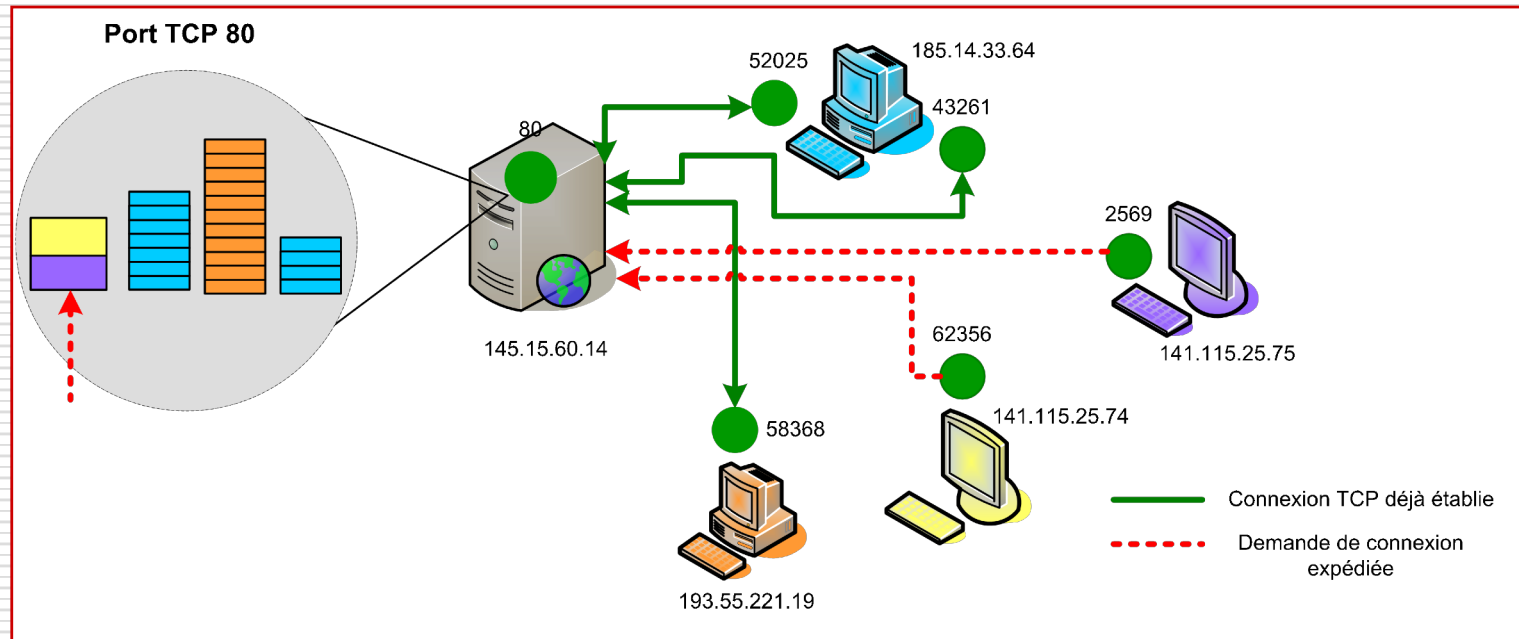
TCP (1/28) Généralités (1/1)

- La valeur ajoutée au service IP est importante :
TCP offre un transfert fiable et régulé de bout en bout sur une connexion entre deux processus s'exécutant sur deux équipements distants.
- Caractéristiques générales :
 - Mode connecté : ==> *trois phases successives :*
 1. *établissement connexion,*
 2. *transfert,*
 3. *libération connexion*
 - Transfert de données fiable et régulé : contrôle d'échange
checksum, numérotation et acquittement, retransmission...
 - Full Duplex :
==> *Echange simultané de deux flux bidirectionnels .*
 - Orienté flot d'octets (« byte stream oriented ») :
==> *TCP gère un flot d'octets sur chaque sens de la connexion*
 - Contrôle de flux et contrôle de congestion :
==> *mécanismes de fenêtres à taille variable*
- Exemples d'applications :
 - Web (HTTP), transferts de fichiers (FTP et dérivés), mails (SMTP), session à distance sécurisée (SSH) ...

TCP (2/28) Gestion des connexions (1/5)

□ Connexion et port TCP :

- Notions très liées dans TCP
 - A chaque port TCP :
 - Une FIFO des demandes de connexions
 - FIFO(s) de messages par connexion établie
- = = > Partage d'un port TCP par plusieurs connexions



TCP (3/28) Gestion des connexions (2/5)

- Ouverture active vs ouverture passive :
 - Les deux entités « utilisateurs » de TCP ont des rôles dissymétriques :
 - le processus initiateur demande à TCP une ouverture de connexion active (en principe une entité client).
 - le processus répondant demande à TCP une ouverture de connexion passive (par exemple une entité serveur).
 - Une demande d'ouverture active, si elle est acceptée côté « passif » va déclencher des échanges protocolaires entre les 2 entités de transport :
 - *Procédure de Handshake en 3 échanges*
- Une machine à états finis (FSM) formalise les échanges protocolaires pour l'établissement et la libération de la connexion (cf. diapo en suivant)

TCP (4/28) Gestion des connexions (3/5)

□ Numérotation des octets

- TCP doit détecter les pertes, duplications, déséquences
== > *Mécanismes de numérotation et d'acquittement des données nécessaires.*
- TCP est "orienté flot d'octets"
== > *Numérotation des octets (données et acquittement).*

□ ISN : *Initial Sequence Number*

- Valeur initiale du compteur d'octets
== > *Produite par un système d'horloges locales est incrémentée toutes les 4 ms par le module TCP d'un équipement (Solution au PB de connexions antérieures à extrémités identiques)*

□ Synchronisation des numéros de séquence :

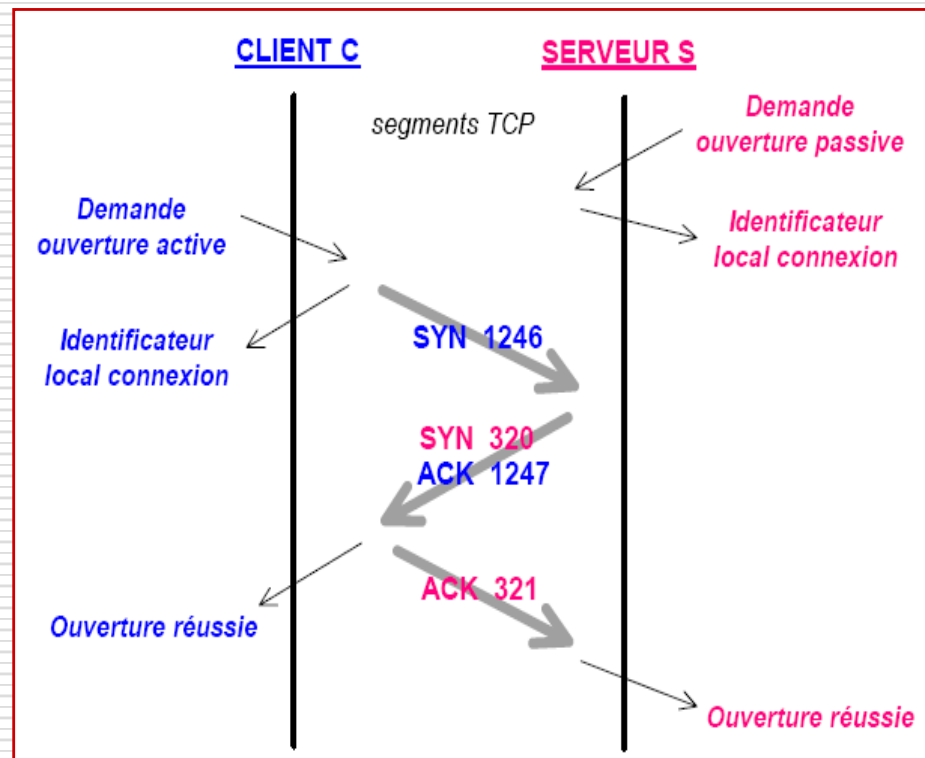
- TCP est full duplex : deux flots d'octets gérés
== > *Nécessité pour chacune des deux entités de connaître l'ISN utilisé par l'autre entité pour la numérotation de son propre flot d'octets.*
- Synchronisation effectuée lors de l'établissement de la connexion == > **utilisation du bit SYN de l'en-tête**

TCP (5/28) Gestion des connexions (4/5)

□ Procédure d'établissement en 3 phases :

■ Procédure de handshake dite « SYN/SYN-ACK/ACK »

1. **C** envoie un segment de demande d'ouverture de connexion. Celui-ci correspond en réalité à une demande de synchronisation (bit SYN) sur le numéro de séquence initial précisé X (ici 1246).
2. **S** reçoit la requête et enregistre le ISN X de **C**. **S** répond par un segment acquittant (bit ACK) le ISN X à la valeur X+1 et précisant une demande de synchronisation (bit SYN) sur son propre ISN Y (ici 320).
3. **C** reçoit le segment, enregistre à son tour le ISN Y de **S** puis répond par un segment acquittant (bit ACK) le ISN Y à la valeur Y+1.

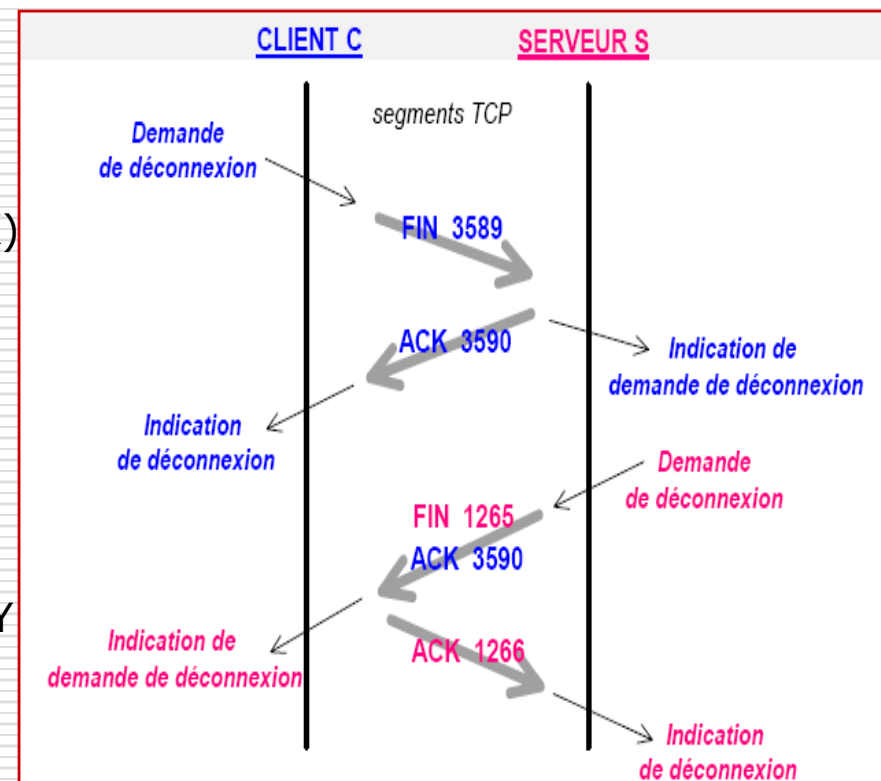


TCP (6/28) Gestion des connexions (5/5)

□ Procédure de clôture en 4 phases :

- Une connexion TCP supporte simultanément deux flots bidirectionnels : elle est libérée lorsque les deux flots sont terminés :

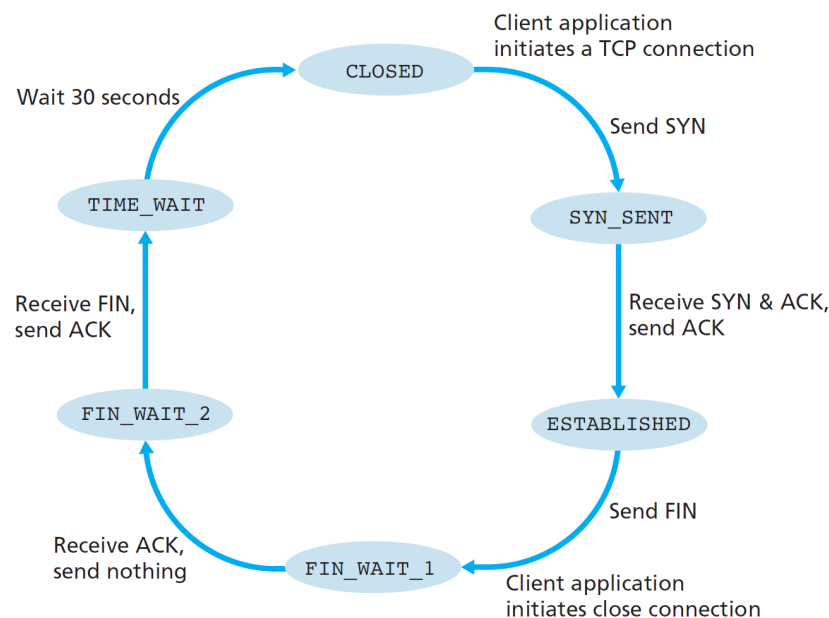
1. Une entité TCP, en principe **C**, envoie un segment de demande de fermeture de connexion (bit FIN). Il indique une fin d'émission de flot et le dernier numéro de séquence X (ici 3589).
2. (a) **S** reçoit la requête et acquitte (bit ACK) immédiatement par X+1 signale à l'application qu'une demande de fin de connexion est en cours.
(b) **S**, à la demande de l'application, envoie à son tour une demande de fermeture de la connexion (bit FIN). Le segment correspondant indique la terminaison du flot, son numéro terminal Y (ici 1265) et acquitte à nouveau par X+1.
3. **C** reçoit le segment et à son tour répond par un segment acquittant par la valeur Y+1.



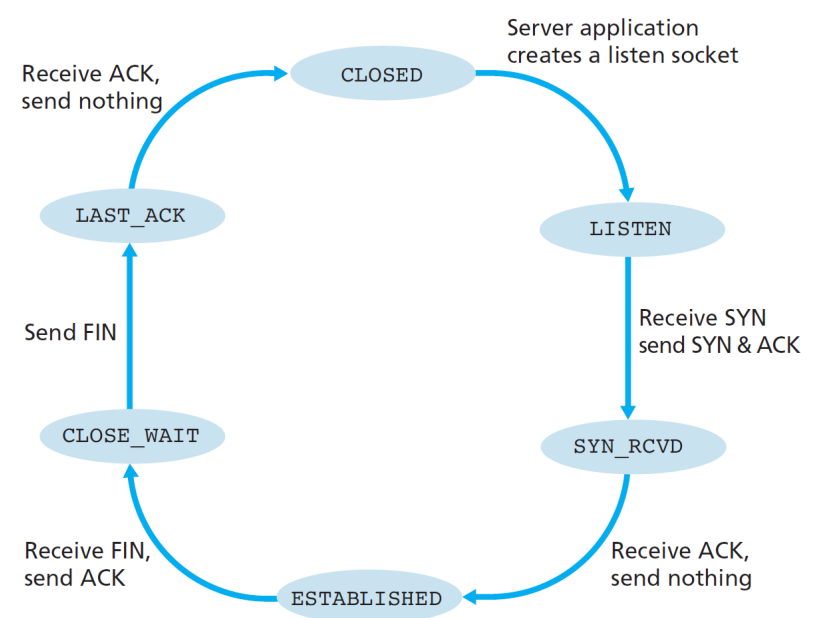
TCP (7/28) Formalisation par FSM (1/1)

□ Machines à états finis

Côté (socket) processus **Client**



Côté (socket) processus **Serveur**



[Fig. Kurose and Ross]

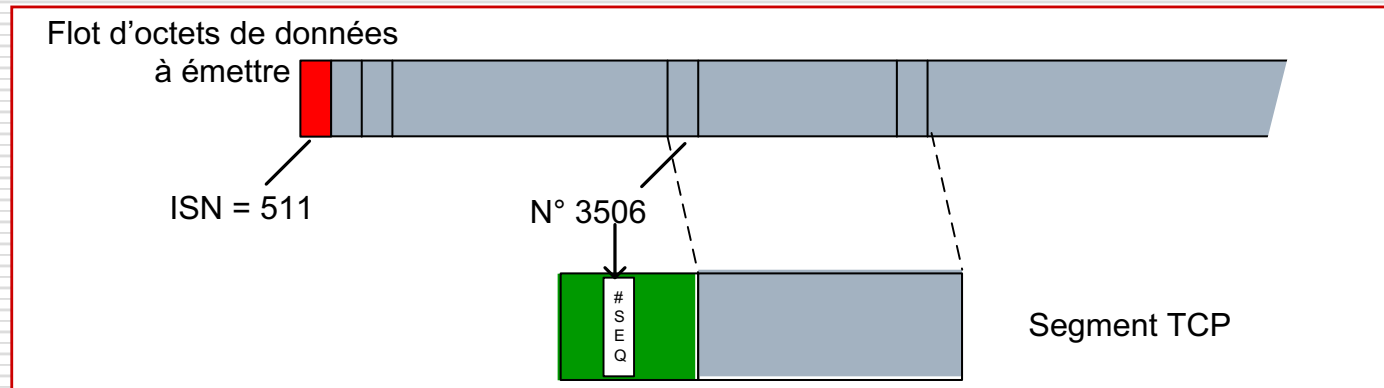
TCP (8/28) Transfert des données (1/14)

- Fiabilisation du transfert de données :
 - Reprise sur erreurs
 - Détection d'erreurs basée sur une somme de contrôle (*Internet checksum*)
 - Retransmissions
 - Gestion de temporisateurs (ARQ)
 - Acquittements positifs et cumulatifs (cf. *Go Back N*)
 - Retransmission individuelle (cf. *Selective Repeat*)
 - Contrôle de flux
 - Fenêtre d'anticipation (*sliding window*) à taille variable
 - Contrôle de congestion

TCP (9/28) Transfert des données (2/14)

□ Segmentation :

- L'unité d'échange d'octets est le segment.
- Un segment TCP possède un en-tête d'au moins 20 octets suivi ou pas d'un groupe d'octets de données du flot à transférer.
- Un segment n'est pas lui-même numéroté
- Si le segment est porteur d'octets de données, il mentionne dans le champ numéro de séquence, le numéro du premier octet porté.
- **MSS** : Maximum Segment Size
 - Taille maximale des données d'un segment.
 - Eventuellement annoncée lors de l'établissement de connexion (option d'en-tête lors d'un message SYN 1)
 - En principe : $MTU - 40$ - par défaut : 536
- Illustration dans la phase de transfert de données, à l'émission :



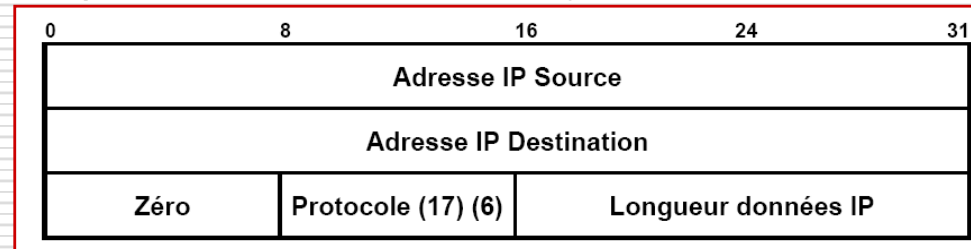
TCP (10/28) Transfert des données (3/14)

□ Intégrité des données et de la destination :

■ Mécanisme de checksum (Idem pour UDP) :

□ Algorithme de calcul (émission) :

1. Le champ contrôle de l'en-tête UDP/TCP est mis à zéro
2. Un pseudo-en-tête IP est rajouté avant l'en-tête UDP/TCP



3. Des bits de bourrage sont éventuellement rajoutés à la fin des données pour obtenir une longueur multiple de 16 bits
4. Le checksum est alors calculé (idem IP) sur la totalité de l'objet obtenu : **pseudo en-tête IP** + **en-tête UDP/TCP** + **données**
5. Le champ contrôle de l'en-tête UDP/TCP reçoit la valeur obtenue
6. Le pseudo en-tête et les bits de bourrage ne sont pas transmis

□ A la réception :

7. Pour vérifier le total contrôle, le récepteur extrait de l'en-tête du datagramme IP qui a acheminé le message de Transport les champs nécessaires à la construction du pseudo en-tête UDP/TCP.
Le calcul et la vérification peuvent alors s'opérer.

TCP (11/28) Transfert des données (4/14)

□ Gestion des acquittements TCP

■ Mécanisme d'acquittement positif :

- Un acquittement n'est envoyé à l'émetteur par le récepteur que si les données émises par le premier sont parvenues au second.
- Un acquittement reçu par un émetteur de données confirme que ces données ont été correctement émises, puis transmises et bien reçues par le destinataire

= = > *détection d'erreur = non réception d'acquittement positif*

■ Mécanisme d'acquittement cumulatif :

- Un acquittement n'est pas envoyé pour chaque octet reçu, mais pour un groupe d'octets reçus
- Les données reçues sont acquittées en précisant le numéro du prochain octet attendu sur la séquence du flot .

= = > *combien d'octets du flot ont été «accumulés» jusqu'à présent*

- On parle de segment reçu « Hors séquence » lorsque le numéro de séquence dont il est porteur est supérieur à celui du prochain octet normalement espéré (stockage sans acquittement spécifique immédiat).

■ Mécanisme d'anticipation :

- L'émetteur peut envoyer des octets suivants dans le flot sans attendre la réception d'acquittement concernant ceux précédemment envoyés

TCP (12/28) Transfert des données (5/14)

□ Gestion des acquittements :

- Mécanisme de superposition (Piggybacking) :
 - TCP ne propose pas des formats de segments différents pour l'échange de données et les acquittements : un même segment peut être porteur de données et d'acquittement
 - = = > *tout segment TCP peut être porteur d'informations relatives aux deux flots associés à une connexion TCP :*
 - L'envoi d'un acquittement pour les données en séquence peut être retardé pour opérer une superposition.
 - L'envoi d'un acquittement pour les données hors séquence est immédiat.

Soit une connexion TCP établie entre **C** et **S** :

- Un segment émis de **C** vers **S** peut être porteur :
 - De données du flux d'octets allant de **C** vers **S** et d'un acquittement portant sur le flux d'octets allant de **S** vers **C**
- Un segment émis de **S** vers **C** peut être porteur :
 - De données du flux d'octets allant de **S** vers **C** et d'un acquittement portant sur le flux allant de **C** vers **S**

TCP (13/28) Transfert des données (6/14)

□ Synthèse :

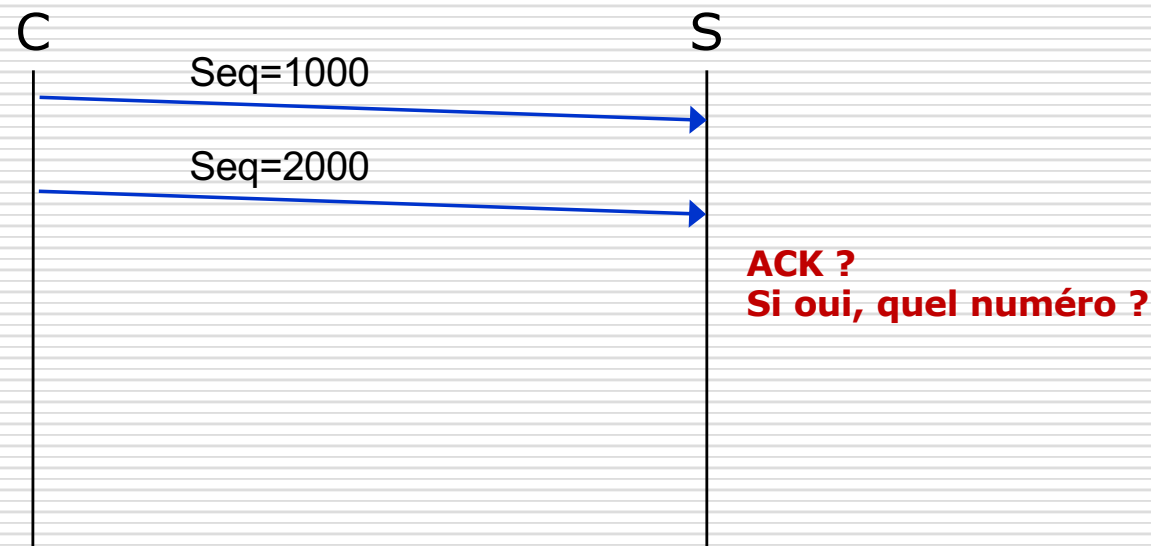
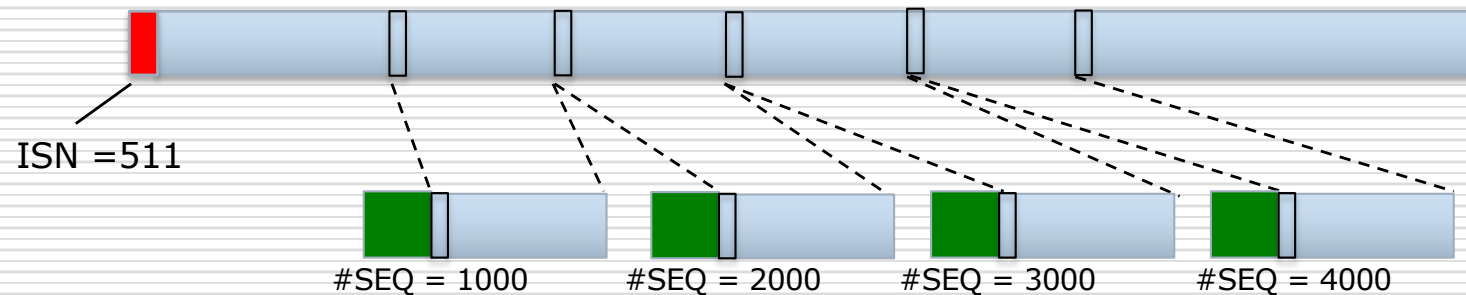
- Côté émetteur / Politique de [retransmission](#) :
 - Un temporisateur d'attente d'acquittement par segment envoyé : **RTO** (Retransmission Time Out)
 - Si un acquittement positif n'a pas été reçu à son expiration : retransmission du segment

- Côté récepteur / Politique d'[envoi des acquittements](#) :
 - Envoi différé (au prochain envoi de données ou au plus à 500 ms)
==> privilégier le *PiggyBacking*
 - Si segment reçu dans l'ordre et tout ce qui précède a été acquitté
 - Envoi immédiat
 - Si segment reçu dans l'ordre et un ACK d'un segment précédent est en attente : l'ensemble est acquitté
 - Si segment reçu est hors séquence : envoi d'un acquittement dupliqué
 - Si segment reçu est un segment manquant attendu : envoi d'un ACK portant sur la séquence complétée

TCP (14/28) Transfert des données (7/14)

□ Exemples :

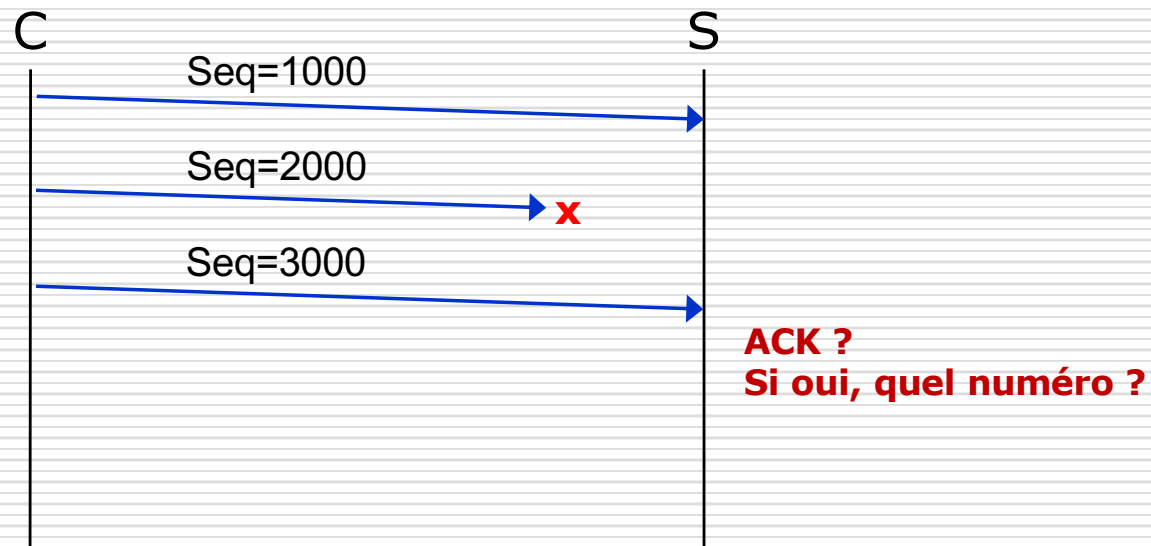
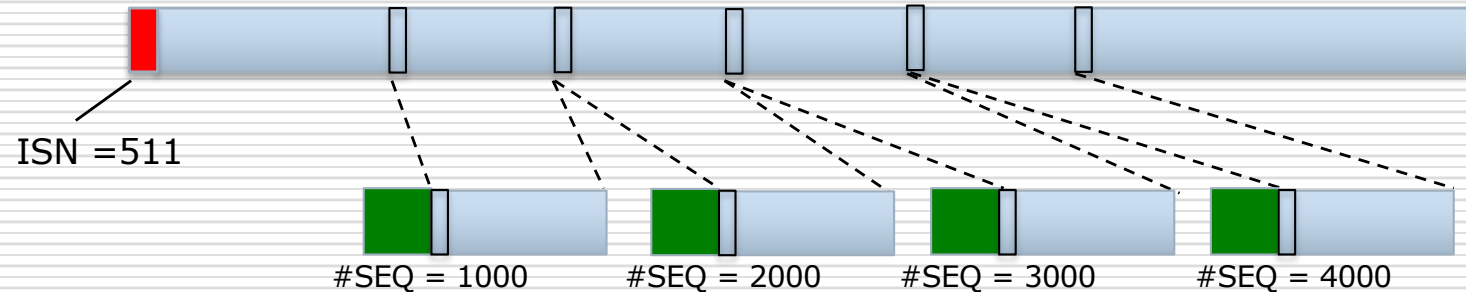
- Flux C --> S



TCP (15/28) Transfert des données (8/14)

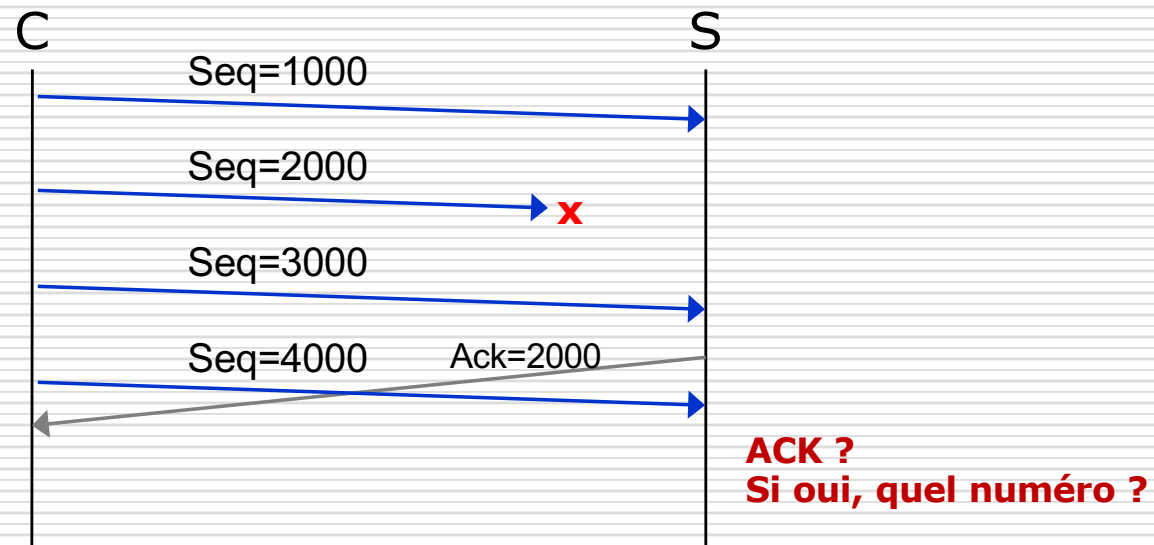
□ Examples :

- Flux C $\rightarrow$ S



□ Examples :

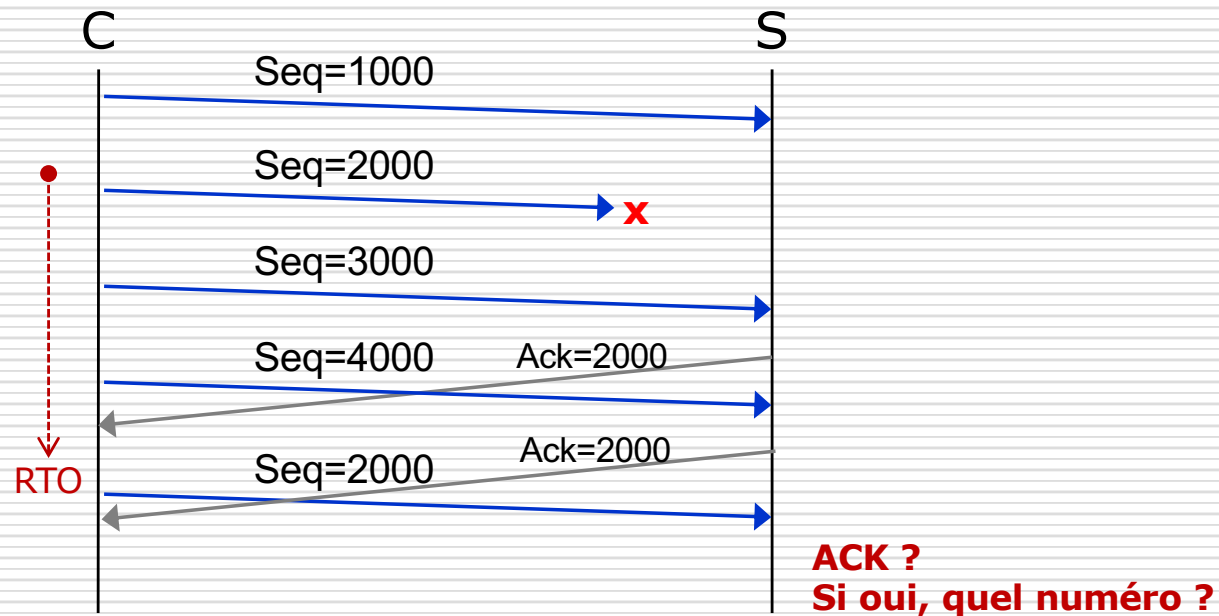
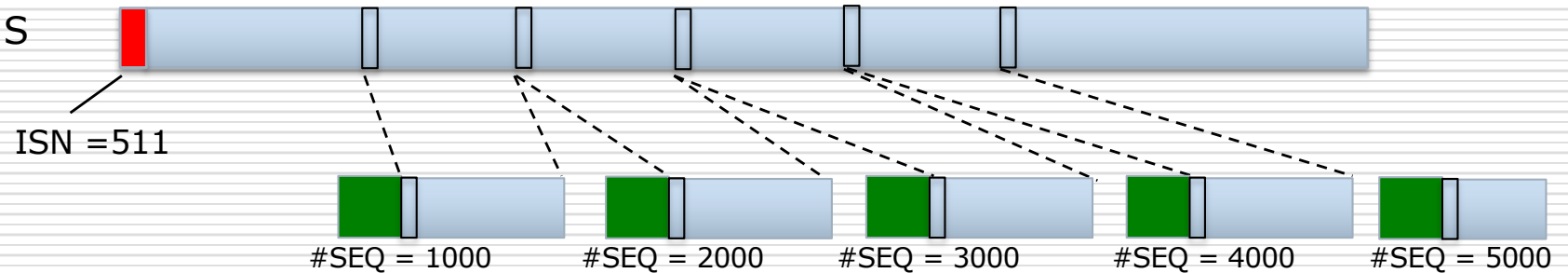
-
- S
- ISN = 511
- #SEQ = 1000
- #SEQ = 2000
- #SEQ = 3000
- #SEQ = 4000
- #SEQ = 5000



TCP (17/28) Transfert des données (10/14)

□ Exemples :

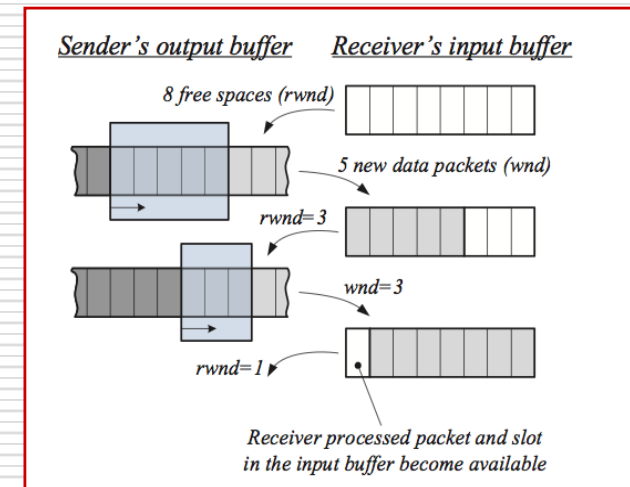
- Flux C --> S



TCP (18/28) Transfert des données (11/14)

□ Contrôle de flux

- Concerne les deux flux de la connexion
- Objectif :
 - Réguler les processus d'émission et de réception de chaque flux
==> contrôler le débit d'émission en fonction des capacités du récepteur (vitesse de traitement, capacité de stockage etc...)
- Mécanisme de fenêtre à taille variable :
 - Un crédit est explicité à l'émetteur par le récepteur dans chaque segment qu'il lui envoie :
il s'agit d'une limite de la taille de la fenêtre d'anticipation (**R_WND**).
La valeur de ce crédit est variable au fil du temps
==> contrôle de flux adaptatif



TCP (19/28) Transfert des données (12/14)

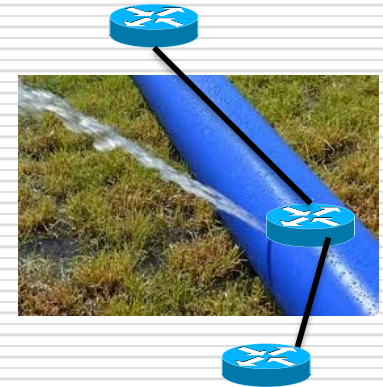
□ Contrôle de congestion

■ Problème :

- De multiples flux TCP (et autres) partagent de façon concurrentes les ressources limitées du réseau

■ Objectif :

- Réguler le débit d'émission selon l'état du réseau pour éviter des pertes de paquets au sein du réseau
==> calculer localement des indicateurs et en déduire une taille de fenêtre de congestion (C_WND)



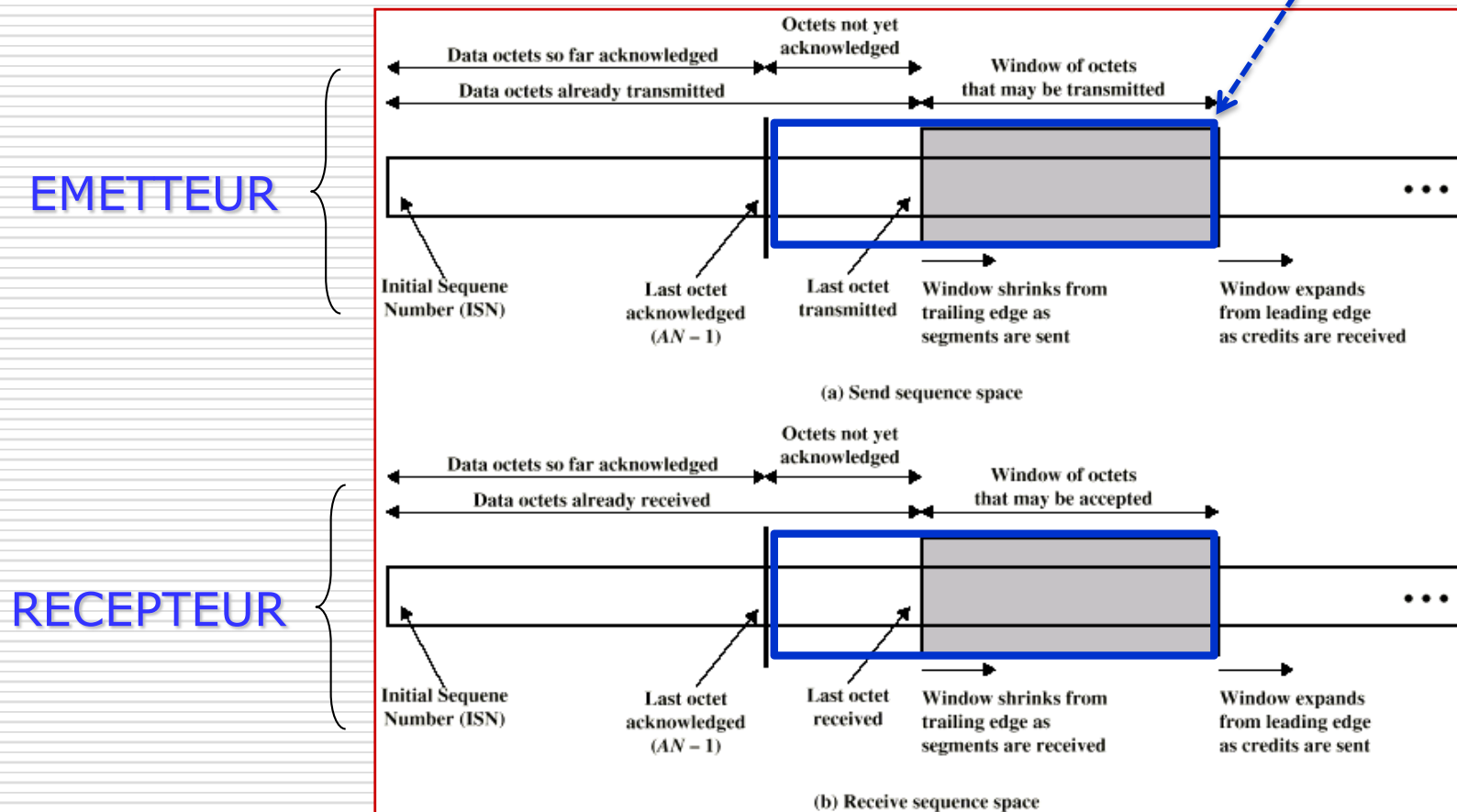
■ Divers algorithmes de régulation : (CF partie II)

- Stratégies réactives
- Stratégies prédictives, impliquant les extrémités
- Stratégies hybrides
- Stratégies impliquant les routeurs

TCP (20/28) Transfert des données (13/14)

□ Gestion des flux et de l'anticipation

$$SND_WND = \text{Min}(R_WND, C_WND)$$

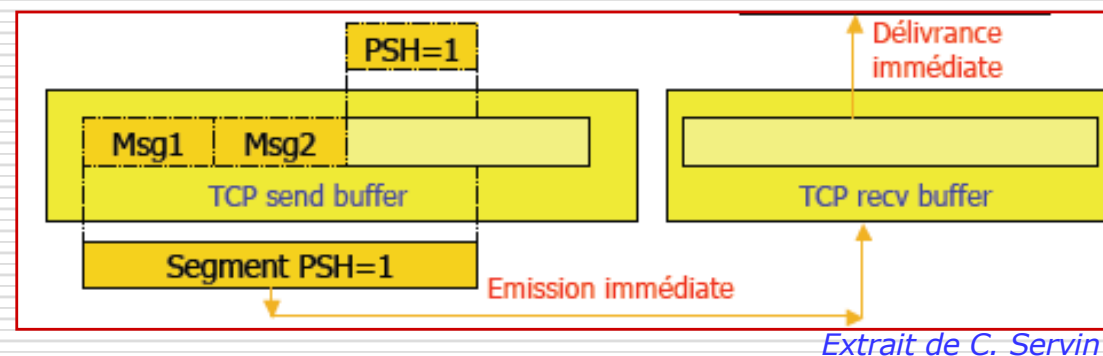


Extrait de W. Stallings

TCP (21/28) Transfert des données (14/14)

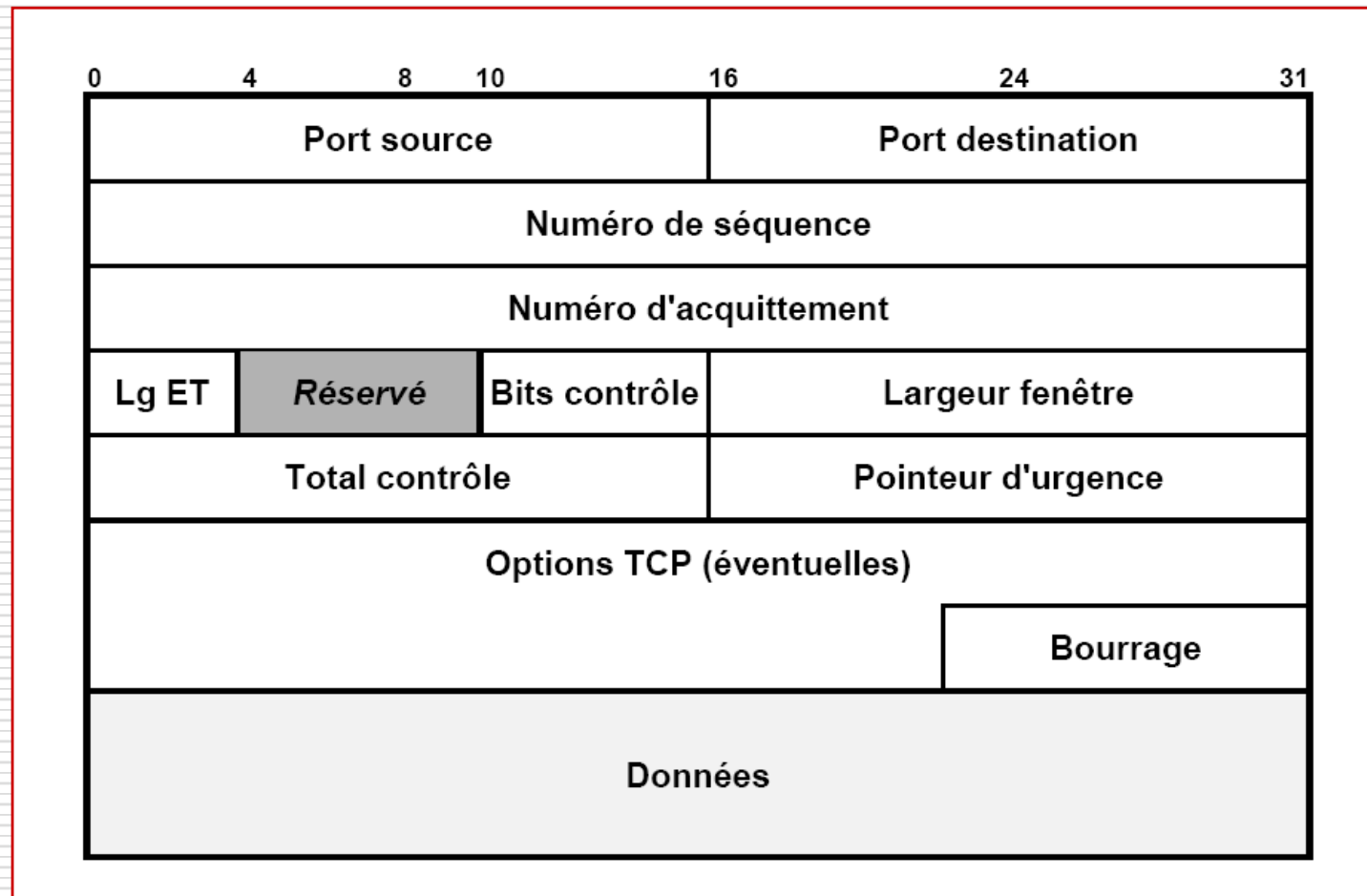
□ Délivrance des données :

- Mécanisme de groupage :
 - Par défaut, TCP attend de remplir complètement le tampon d'émission pour constituer un segment
= = > optimisation de la transmission
 - Le bit PSH positionné à 1 modifie ce comportement par défaut et provoque une transmission et une remise immédiate des données à l'application.



TCP (22/28) Protocole (1/7)

□ Format d'un segment :



TCP (23/28) Protocole (2/7)

□ Partie fixe :

- Champs Port Source et Port Destination (2x16 bits) :
 - Références des ports TCP qui identifient les processus d'application aux extrémités de la connexion de transport.
- Champ Numéro de Séquence (32 bits) :
 - Numéro du premier octet de données transmis le segment. Il correspond à la position du segment dans le flux de l'émetteur.
 - Lors de l'établissement d'une connexion, ce champ contient une valeur initiale (ISN : Initial Sequence Number) attribuée par l'émetteur. Il aura la valeur ISN+1 pour le premier segment de données transmis par l'émetteur.
- Champ Numéro d'Acquittement (32 bits) :
 - Numéro de séquence du prochain octet attendu par l'émetteur de ce message. Il fait référence au flux du récepteur
 - Cette valeur est toujours transmise une fois la connexion établie.
- Champ Longueur En-Tête (4 bits) :
 - Longueur totale (partie fixe et options) de l'en-tête du segment TCP exprimée par un multiple de 32 bits.

TCP (24/28) Protocole (3/7)

□ Partie fixe :

■ Champ Réservé (4 bits)

■ Champ Bits ECN (2 bits)

- Utilisés uniquement si ECN (Explicit Congestion Notification) est mise en œuvre, sinon 00
- Spécifiés dans [RFC 3168], si positionné à 1 :
 - **CWR** : l'émetteur a réduit de moitié sa fenêtre de congestion
 - **ECN-Echo** : le récepteur doit réduire sa fenêtre de congestion de moitié

■ Champ Bits de Contrôle (6/8 bits) :

- Indication du rôle et du contenu du segment, si positionné à 1 :
 - **URG** : le champ pointeur d'urgence doit être considéré
 - **ACK** : le champ numéro d'acquittement doit être considéré
 - **PSH** : les données reçues doivent être transmises immédiatement.
 - **RST** : fermeture de la connexion à cause d'une erreur irrécupérable (réinitialisation attendue).
 - **SYN** : ouverture de connexion, demande de synchronisation sur un ISN.
 - **FIN** : terminaison du flux à émettre, donc fin de connexion pour le flot de l'émetteur.

TCP (25/28) Protocole (4/7)

□ Partie fixe :

- Champ Largeur de Fenêtre (16 bits)
 - Nombre d'octets que le récepteur peut accepter.
Ce champ permet de gérer le contrôle de flux.
- Champ Totale Contrôle (16 bits) :
 - Checksum sur le segment TCP.
- Champ Pointeur d'Urgence (8 bits) :
 - Position des données à traiter en priorité.

TCP (26/28) Protocole (5/7)

□ Options :

□ Taille des segments MSS (4 octets) [RFC 793] :

- Permet lors de l'établissement de connexion (SYN = 1) d'annoncer la MSS admise
- Format :

Code = 2 1 Ø	Lgueur = 4 1 Ø	MSS 2 Ø
------------------------	--------------------------	-------------------

□ Estampille horaire (*Timestamps*)(10 octets) [RFC 793] :

- Permet d'indiquer l'heure d'émission d'un segment et de provoquer un écho indiquant l'heure de réception de celui-ci (calcul précis du RTT)
- Format :

Code = 8 1 Ø	Lgueur = 10 1 Ø	Horodatage 4 Ø	Echo Horodatage 4 Ø
------------------------	---------------------------	--------------------------	-------------------------------

□ Taille fenêtre transmission (*Window Scale*)(4 octets) [RFC 1323] :

- Permet d'adapter TCP aux réseaux haut débit ou à fort taux de latence pour permettre une continuité du flux de données (sinon arrêt fréquent en attente d'acquittement) en proposant de multiplier par un multiple de 2 la taille de W (au plus 2^{16} dans TCP car champ sur 2 octets)
- Format :

Code = 3 1 Ø	Lgueur = 3 1 Ø	Décalage 0 à 14 1 Ø
------------------------	--------------------------	-------------------------------

TCP (27/28) Protocole (6/7)

□ Options :

□ Négociation de SACK (2 octets) [RFC 2018] :

- Permet lors de l'établissement de connexion (SYN = 1) de négocier l'utilisation d'acquittements sélectifs (option ci-après).

- Format :

Code = 4 1 Ø	Lgueur = 2 1 Ø
-----------------	-------------------

□ Acquittement Sélectif (SACK) (n octets) [RFC 2018] :

- Permet d'acquitter une liste de blocs d'octets reçus hors séquence.

- Format :

Code = 5 1 Ø	Lgueur = n 1 Ø	Début Bloc i 4 Ø	Fin Boc i 4 Ø
-----------------	-------------------	---------------------	------------------

*

□ Alignement d'en-tête (1 octet) [RFC 793] :

- Permet d'aligner le prochain champ option sur un début de mot de 32 bits (le champ option de TCP peut contenir au plus 10 mots).

- Format :

Code = 1 1 Ø

□ Fin de liste (1 octet) [RFC 793] :

- Permet d'indiquer la fin du champ options et donc le début d'un éventuel bourrage.

- Format :

Code = 0 1 Ø

TCP (28/28) Protocole (7/7)

□ Options :

□ Authentication (TCP-AO)(>3 octets) [RFC 5925] :

- Permet un niveau d'authentification à base de clés (si par exemple IPSec ou TLS non disponibles).
- Format :

Code = 29 1 Ø	Lgueur > 3 1 Ø	Infos Authentification n Ø
-------------------------	-----------------------------	--------------------------------------

□ TCP-Fast Open Cookie (TFO) (n octets) [RFC 7413] :

- Permet le transport de données au sein des segments d'établissement de connexions SYN et SYN/ACK et leur délivrance aux applications si elles sont compatibles à ce mécanisme. L'option permet l'échange initial d'un *cookie* du serveur vers le client.
- Format :

Code = 34 1 Ø	Lgueur > 3 1 Ø	Cookie (0,4 ou 16) Ø
-------------------------	-----------------------------	--------------------------------

□ Options MPTCP (Multipath TCP)[RFC 6824] :

- Plusieurs options qui permettent de négocier l'utilisation de MP-TCP et, si tel est le cas, de gérer plusieurs connexions TCP s'appuyant sur des chemins différents, entre deux processus applicatifs distants, comme s'ils n'utilisaient qu'une seule et même connexion TCP.